

Trabalho Prático | DGT2811

Desenvolvimento Back-End Corporativo com Java E Cloud

*Material de **orientações** para desenvolvimento do **trabalho prático** da disciplina DGT2811
Desenvolvimento Back-End Corporativo com Java e Cloud*

Implementação de sistema cadastral com interface Web, baseado nas tecnologias de Servlets, JPA e JEE.

Objetivos do trabalho prático

1. Implementar persistência com base em JPA.
2. Implementar regras de negócio na plataforma JEE, através de EJBs.
3. Implementar sistema cadastral Web com base em Servlets e JSPs.
4. Utilizar a biblioteca Bootstrap para melhoria do design.
5. No final do exercício, o aluno terá criado todos os elementos necessários para exibição e entrada de dados na plataforma Java Web, tornando-se capacitado para lidar com contextos reais de aplicação.

 **O trabalho prático deve ser feito individualmente.**

Materiais necessários para o trabalho prático

- SQL Server, com o banco de dados gerado em prática anterior (loja).
 - JDK e IDE NetBeans.
 - Navegador para Internet, como o Chrome.
 - Banco de dados SQL Server com o Management Studio.
 - Equipamentos:
- Computador com acesso à Internet.

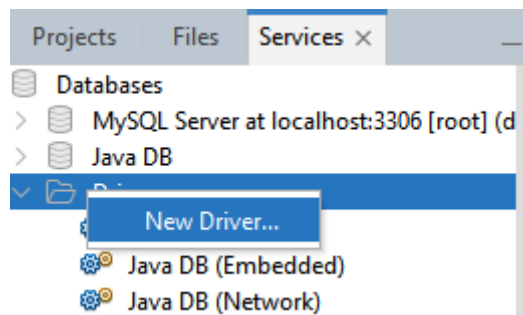
- JDK e IDE NetBeans.
- Banco de dados SQL Server.
- Navegador de Internet instalado no computador.

Desenvolvimento do trabalho prático

Vamos colocar a **mão na massa**?! Siga as **instruções** abaixo para desenvolvimento deste trabalho.

1º Procedimento | Camadas de Persistência e Controle

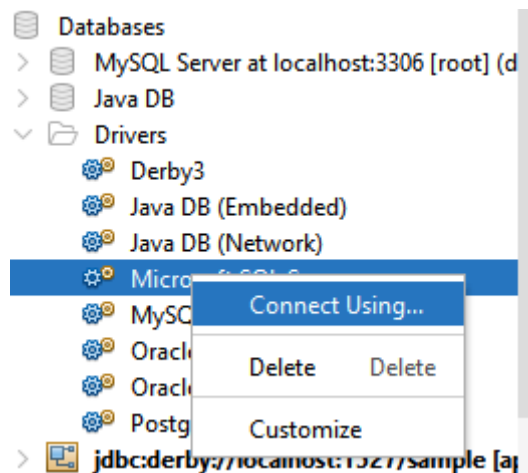
1. Configurar a conexão com SQL Server via NetBeans e o pool de conexões no GlassFish Server 6.2.1:
 - Na aba de **Serviços**, divisão **Banco de Dados**, clique com o botão direito em **Drivers** e escolha **Novo Driver**.



- b. Na janela que se abrirá, clicar em **Add** (Adicionar), selecionar o arquivo **mssql-jdbc-12.2.0.jre8.jar**, que é parte do arquivo **zip** encontrado no endereço seguinte, e finalizar com **Ok**

<https://learn.microsoft.com/pt-br/sql/connect/jdbc/download-microsoft-jdbc-driver-for-sql-server?view=sql-server-ver16>

- c. O reconhecimento será automático, e podemos definir uma conexão com o clique do botão direito sobre o driver e escolha de **Conectar Utilizando**.



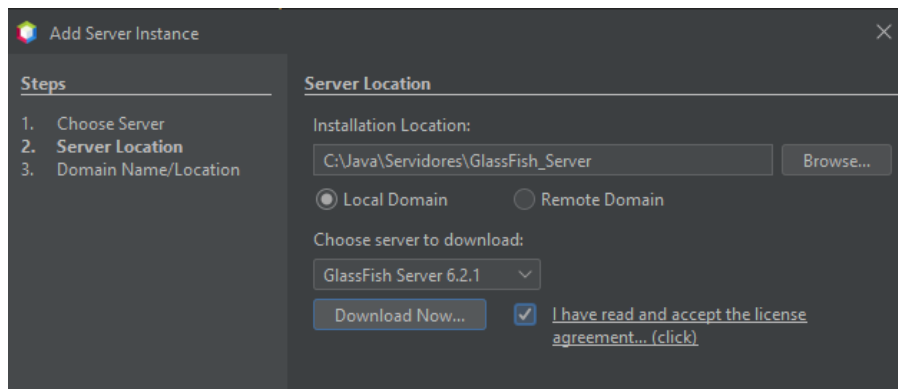
d. Para os campos **database**, **user** e **password**, utilizar o valor **loja**, de acordo com os elementos criados em exercício anterior sobre a criação do banco de dados de exemplo, marcando também a opção **Lembrar Senha**.

e. Para o campo JDBC URL deve ser utilizada a seguinte expressão:

jdbc:sqlserver://localhost:1433;databaseName=loja;encrypt=true;trustServerCertificate=true;

f. Clicar em **Testar Conexão** e, estando tudo certo, **Finalizar**.

g. Na divisão **Servidores**, verificar se o GlassFish 6.2.1 (ou posterior) está instalado, e caso não esteja, adicionar o servidor, via clique com o botão direito e escolha da opção **Add Server**, efetuando o download a partir da própria janela que se abrirá



h. Copiar o arquivo **mssql-jdbc-12.2.0.jre8.jar** para o subdiretório **lib**, a partir do diretório de base do **GlassFish**.

i. Iniciar o servidor GlassFish a partir do NetBeans.

j. Através da linha de comando, executar o comando **asadmin**, no diretório **bin** do **GlassFish**.

k. No **prompt do asadmin**, executar o comando apresentado a seguir

create-jdbc-connection-pool

--datasourceclassname com.microsoft.sqlserver.jdbc.SQLServerDataSource

--restype javax.sql.DataSource

--property

driverClass=com.microsoft.sqlserver.jdbc.SQLServerDriver:portNumber=1433:password=loja:user=loja:serverName=localhost:databaseName=loja:trustServerCertificate=true:URL="jdbc\\:sqlserver\\://localhost\\:1433\\;databaseName\\=loja\\;encrypt\\=true\\;trustServerCertificate\\=true\\";

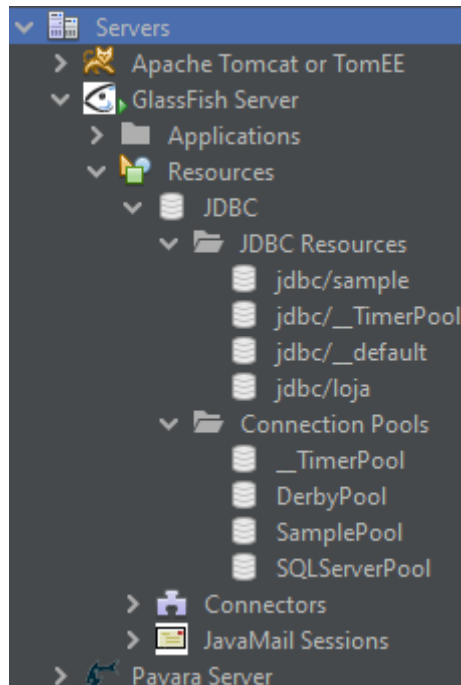
- l. Será solicitado o identificador do pool, que será **SQLServerPool**.
- m. Testar o pool de conexões através do comando apresentado a seguir:

ping-connection-pool SQLServerPool

n. Obtendo sucesso na operação, criar o registro **JNDI**, ainda no **asadmin**, através do comando apresentado a seguir:

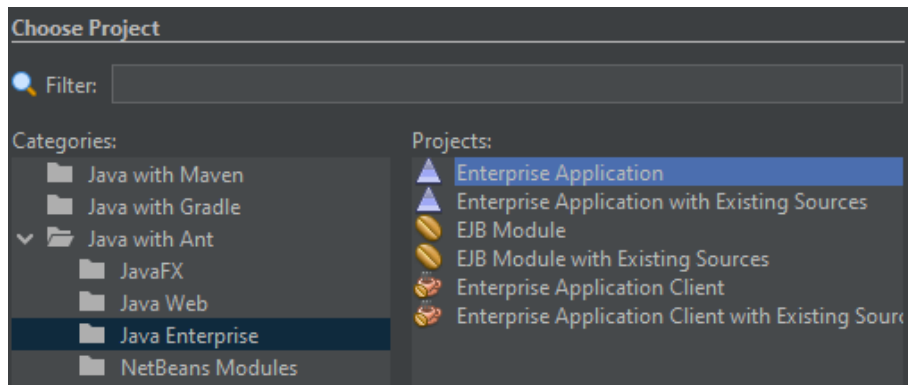
create-jdbc-resource --connectionpoolid SQLServerPool jdbc/loja

o. Atualizar o servidor no ambiente do NetBeans e verificar se tudo foi gerado corretamente



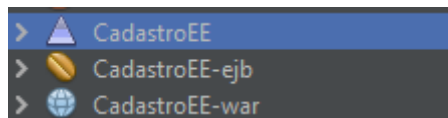
2. Criar o aplicativo corporativo no NetBeans:

- a. Criar um projeto do tipo **Ant..Java Enterprise..Enterprise Application**.



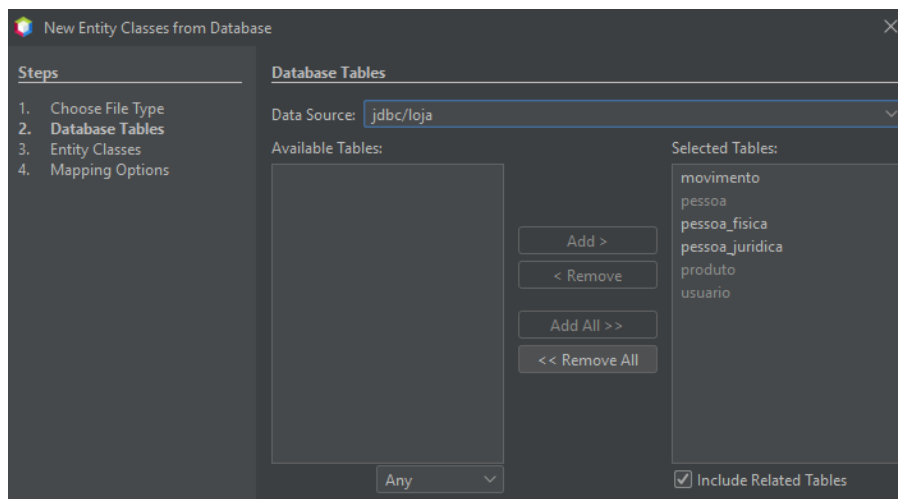
b. Adotar o nome **CadastroEE**, com escolha do servidor **GlassFish**, além de plataforma **Jakarta JEE 8**.

c. Serão gerados três projetos, onde o principal encapsula o arquivo EAR, tendo os outros dois, **CadastroEE-ejb** e **CadastroEE-war**, como projetos dependentes, relacionados aos elementos JPA, JEE e Web.



3. Definir as camadas de persistência e controle no projeto **CadastroEE-ejb**.

- Criar as **entidades JPA** através de **New Entity Classes from Database**.
- Selecionar **jdbc/loja** como **Data Source**, e selecionar todas as tabelas.



c. No passo seguinte, definir o pacote como **cadastroee.model**, além de marcar a opção para criação do arquivo **persistence.xml**.

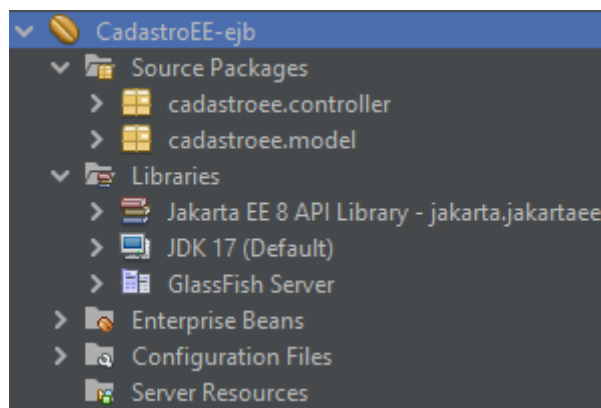
d. Em seguida, adicionar os componentes EJB ao projeto, através da opção **New Session Beans for Entity Classes**.

e. Selecionar **todas as entidades**, marcar a geração da **interface local**, além de definir o nome do pacote como **cadastroee.controller**.

f. Serão gerados todos os **Session Beans**, com o sufixo **Facade**, bem como as **interfaces**, com o sufixo **FacadeLocal**.

4. Efetuar pequenos acertos no projeto, para uso do Jakarta:

- a. Adicionar a biblioteca **Jakarta EE 8 API** ao projeto **CadastroEE-ejb**.
- b. Criados os componentes e ajustadas as bibliotecas, o projeto deverá ficar como apresentado a seguir.



c. Modificar **TODAS** as importações de pacotes **javax** para **jakarta**, em todos os arquivos do projeto **CadastroEE-ejb**.

```
5 package cadastroee.model;  
6  
7 import java.io.Serializable;  
8 import java.util.Collection;  
9 import jakarta.persistence.Basic;  
10 import jakarta.persistence.CascadeType;  
11 import jakarta.persistence.Column;
```

d. Na entidade **Produto**, mudar o tipo do atributo **precoVenda** para **Float** no lugar de **BigDecimal**.

e. Modificar o arquivo **persistence.xml** para o que é apresentado a seguir:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<persistence version="1.0"
```

```
xmlns="http://java.sun.com/xml/ns/persistence"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
```

```
http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd">
```

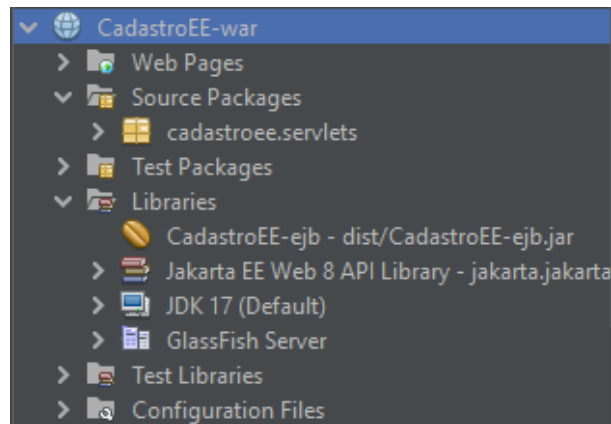
```
<persistence-unit name="CadastroEE-ejbPU" transaction-type="JTA">  
  <jta-data-source>jdbc/loja</jta-data-source>  
  <exclude-unlisted-classes>>false</exclude-unlisted-classes>  
  <properties/>  
</persistence-unit>  
</persistence>
```

5. Criar um **Servlet** de teste no projeto **CadastroEE-war**

- a. Utilizar o clique do botão direito e escolha da opção **New..Servlet**
- b. Definir o nome do Servlet como **ServletProduto**, e nome do pacote como **cadastroee.servlets**
- c. Marcar opção **Add information to deployment descriptor**, algo que ainda é necessário quando o GlassFish 6 é utilizado
- d. Adicionar, no código do Servlet, a referência para a interface do EJB
@EJB
ProdutoFacadeLocal facade;
- e. Modificar a resposta do Servlet, utilizando o **facade** para recuperar os dados e apresentá-los na forma de lista HTML

6. Efetuar novos acertos no projeto, para uso do Jakarta:

- Adicionar a biblioteca **Jakarta EE Web 8 API** ao projeto **CadatroEE-war**
- Criado o Servlet e ajustadas as bibliotecas, o projeto deverá ficar como apresentado a seguir:



c. Modificar **TODAS** as importações de pacotes **javax** para **jakarta**, em todos os arquivos do projeto **CadastroEE-war**

d. Modificar o arquivo **web.xml** para o que é apresentado a seguir:

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app version="4.0" xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd">

  <servlet>

    <servlet-name>ServletProduto</servlet-name>

    <servlet-class>cadastroee.servlets.ServletProduto</servlet-class>

  </servlet>

  <servlet>

    <servlet-name>ServletProdutoFC</servlet-name>

    <servlet-class>cadastroee.servlets.ServletProdutoFC

  </servlet-class>

  </servlet>

  <session-config>

    <session-timeout>30</session-timeout>

  </session-config>

</web-app>
```

7. Executar o projeto:

- A execução deve ser efetuar com o uso de **Run** ou **Deploy** no projeto principal (**CadastroEE**), simbolizado por um triângulo
- Acessar o endereço a seguir, para testar o Servlet **http://localhost:8080/CadastroEE-war/ServletProduto**
- Tendo alimentado a base via SQL Server Management Studio, ou pela aba de serviços do NetBeans, deve ser obtida uma saída como a seguinte:



Resultados esperados

1. É importante que o código seja organizado.
2. Outro ponto importante é explorar as funcionalidades oferecidas pelo NetBeans para melhoria da produtividade.
3. Nesse exercício, é esperado que o estudante demonstre as habilidades básicas para a construção de aplicativos Web na plataforma JEE.

Relatório discente de acompanhamento

Os Relatórios de Práticas deverão ser confeccionados em arquivo no formato PDF, com a Logo da Universidade, nome do Campus, nome do Curso, nome da Disciplina, número da Turma, semestre letivo, nome dos integrantes da Prática. Além disso, o projeto deve ser armazenado em um repositório no GIT e o respectivo endereço deve constar na documentação. A documentação do projeto deve conter:

1. Título da Prática;
2. Objetivo da Prática;
3. Todos os códigos solicitados neste roteiro de aula;

4. Os resultados da execução dos códigos também devem ser apresentados;
5. Análise e Conclusão:
 - Como é organizado um projeto corporativo no NetBeans?
 - Qual o papel das tecnologias JPA e EJB na construção de um aplicativo para a plataforma Web no ambiente Java?
 - Como o NetBeans viabiliza a melhoria de produtividade ao lidar com as tecnologias JPA e EJB?
 - O que são Servlets, e como o NetBeans oferece suporte à construção desse tipo de componentes em um projeto Web?
 - Como é feita a comunicação entre os Servlets e os Session Beans do pool de EJBs?

2º Procedimento | Interface Cadastral com Servlet e JSPs

1. Criar um Servlet com o nome **ServletProdutoFC**, no projeto **CadastroEE-war**:
 - Utilizar o padrão **Front Controller**
 - Adicionar uma referência para **ProdutoFacadeLocal**, utilizando o nome **facade** para o atributo
 - Apagar o conteúdo interno do método **processRequest**, e efetuar nele as modificações seguintes
 - Capturar o parâmetro **acao** a partir do **request**, o qual poderá assumir os valores **listar**, **incluir**, **alterar**, **excluir**, **formIncluir** e **formAlterar**
 - Definir a variável **destino**, contendo o nome do JSP de apresentação, que terá os valores **ProdutoDados.jsp**, para acao valendo formAlterar ou formIncluir, ou **ProdutoLista.jsp**, para as demais opções
 - Para o valor **listar**, adicionar a **listagem de produtos** como atributo da requisição (**request**), com a consulta efetuada via facade
 - Para o valor **formAlterar**, capturar o **id** fornecido como parâmetro do request, consultar a **entidade** via facade, e adicioná-la como atributo da requisição (**request**)

- Para o valor **excluir**, capturar o **id** fornecido como parâmetro do request, remover a entidade através do facade, e adicionar a **listagem de produtos** como atributo da requisição (**request**)
- Para o valor **alterar**, capturar o **id** fornecido como parâmetro do request, consultar a entidade através do facade, preencher os demais campos com os valores fornecidos no request, alterar os dados via facade e adicionar a **listagem de produtos** como atributo da requisição (**request**)
- Para o valor **incluir**, instanciar uma entidade do tipo Produto, preencher os campos com os valores fornecidos no request, inserir via facade e adicionar a **listagem de produtos** como atributo da requisição (**request**)
- Ao final redirecionar para **destino** via **RequestDispatcher**, obtido a partir do objeto request

2. Criar a página de consulta, com o nome **ProdutoLista.jsp**

- Incluir um link para **ServletProdutoFC**, com acao **formIncluir**, voltado para a abertura do formulário de inclusão.
- Definir uma tabela para apresentação dos dados.
- Recuperar a **lista de produtos** enviada pelo Servlet.
- Para cada elemento da lista, apresentar id, nome, quantidade e preço como células da tabela.
- Criar, também, de forma dinâmica, links para **alteração** e **exclusão**, com a chamada para ServletProdutoFC, passando as ações corretas e o id do elemento corrente.
- Organizar o código para obter uma página como a seguinte.



3. Criar a página de cadastro, com o nome **ProdutoDados.jsp**

- Definir um formulário com envio para **ServletProdutoFC**, modo **post**.
- Recuperar a **entidade** enviada pelo Servlet.

- Definir a variável **acao**, com valor **incluir**, para entidade nula, ou **alterar**, quando a entidade é fornecida.
- Incluir um campo do tipo **hidden**, para envio do valor de **acao** .
- Incluir um campo do tipo **hidden**, para envio do **id**, apenas quando o valor de **acao** for **alterar**.
- Incluir os campos para nome, quantidade e preço de venda, preenchendo os dados quando a entidade é fornecida.
- Concluir o formulário com um botão de envio, com o texto adequado para as situações de inclusão ou alteração de dados .
- Organizar o código para obter uma página como a seguinte.



The screenshot shows a web browser window with the address bar displaying `http://localhost:8080/CadastroEE-war/ServletProdutoFCi`. The page title is "JSP Page". The main content area has the heading "Dados do Produto". Below the heading, there are three input fields: "Nome:", "Quantidade:", and "Preço de Venda:". Below these fields is a button labeled "Adicionar Produto".

4. Testar as funcionalidades do sistema:

- Listar os produtos com a chamada para o endereço seguinte:
`http://localhost:8080/CadastroEE-war/ServletProdutoFC?acao=listar`
- Efetuar uma inclusão a partir do link da tela de listagem
- Efetuar uma alteração a partir do link dinâmico da listagem
- Efetuar uma exclusão a partir do link dinâmico da listagem

✅ Resultados esperados

1. É importante que o código seja organizado.
2. Outro ponto importante é explorar as funcionalidades oferecidas pelo NetBeans para melhoria da produtividade.
3. Nesse exercício, é esperado que o estudante demonstre habilidade para utilizar Servlets e JSPs na construção de interfaces cadastrais para Web.

Relatório discente de acompanhamento

Os Relatórios de Práticas deverão ser confeccionados em arquivo no formato PDF, com a Logo da Universidade, nome do Campus, nome do Curso, nome da Disciplina, número da Turma, semestre letivo, nome dos integrantes da Prática. Além disso, o projeto deve ser armazenado em um repositório no GIT e o respectivo endereço deve constar na documentação. A documentação do projeto deve conter:

1. Título da Prática;
2. Objetivo da Prática;
3. Todos os códigos solicitados neste roteiro de aula;
4. Os resultados da execução dos códigos também devem ser apresentados;
5. Análise e Conclusão:
 - Como funciona o padrão Front Controller, e como ele é implementado em um aplicativo Web Java, na arquitetura MVC?
 - Quais as diferenças e semelhanças entre Servlets e JSPs?
 - Qual a diferença entre um redirecionamento simples e o uso do método forward, a partir do RequestDispatcher? Para que servem parâmetros e atributos nos objetos HttpRequest?

3º Procedimento | Melhorando o Design da Interface

1. Incluir as bibliotecas do framework **Bootstrap** nos arquivos **ProdutoLista.jsp** e **ProdutoDados.jsp**
 - Visitar o site do **BootStrap**, no endereço <https://getbootstrap.com/>
 - Rolar para baixo até encontrar a inclusão via **CDN**



c. Clicar no botão para cópia do link **CSS** e colar na divisão **head** de cada uma das páginas JSP.

d. Clicar no botão para cópia do link para a biblioteca **Java Script** e colar na divisão **head** de cada uma das páginas JSP

2. Modificar as características de **ProdutoLista.jsp**

- Adicionar a classe **container** ao body.
- Adicionar as classes **btn**, **btn-primary** e **m-2** no link de inclusão.
- Adicionar as classes **table** e **table-striped** na tabela.
- Adicionar a classe **table-dark** ao thead.
- Adicionar as classes **btn**, **btn-primary** e **btn-sm** no link de alteração.
- Adicionar as classes **btn**, **btn-danger** e **btn-sm** no link de exclusão.
- Ajustar as características para obter o design apresentado a seguir.

Listagem de Produtos

Novo Produto				
#	Nome	Quantidade	Preço de Venda	Opções
1	Banana	100	5.0	Alterar Excluir
3	Laranja	500	2.0	Alterar Excluir
4	Manga	800	4.0	Alterar Excluir

3. Modificar as características de **ProdutoDados.jsp**

- Adicionar a classe **container** ao body.
- Encapsule cada par label / input em **div** com classe **mb-3**.
- Adicionar a classe **form** ao formulário.
- Adicionar a classe **form-label** em cada label.

- Adicionar a classe **form-control** em cada input.
- Adicionar as classes **btn** e **btn-primary** ao botão de inclusão.
- Ajustar as características para obter o design apresentado a seguir.

Dados do Produto

Nome:

Quantidade:

Preço de Venda:

Resultados esperados

1. É importante que o código seja organizado.
2. Nesse exercício, é esperado que o estudante demonstre habilidade para incluir o framework Bootstrap e utilizá-lo na melhoria do design.

Relatório discente de acompanhamento

Os Relatórios de Práticas deverão ser confeccionados em arquivo no formato PDF, com a Logo da Universidade, nome do Campus, nome do Curso, nome da Disciplina, número da Turma, semestre letivo, nome dos integrantes da Prática. Além disso, o projeto deve ser armazenado em um repositório no GIT e o respectivo endereço deve constar na documentação. A documentação do projeto deve conter:

1. Título da Prática;
2. Objetivo da Prática;
3. Todos os códigos solicitados neste roteiro de aula;
4. Os resultados da execução dos códigos também devem ser apresentados;

5. Análise e Conclusão:

- Como o framework Bootstrap é utilizado?
- Por que o Bootstrap garante a independência estrutural do HTML?
- Qual a relação entre o Bootstrap e a responsividade da página?

Observações

Pré-requisitos:

1. Os estudantes precisam instalar o JDK e o NetBeans;
2. Também é necessário instalar o SQL Server e criar o banco de dados que foi solicitado na **Prática 2 – Vamos manter as informações**;
3. Acesso à Internet para utilizar o repositório CDN do Bootstrap.

Referência Bibliográfica:

1. <https://stensenme.blob.core.windows.net/hmlgreph/00212ti/00905/index.html>
2. <https://stensenme.blob.core.windows.net/hmlgreph/00212ti/00965/index.html>
3. <https://stensenme.blob.core.windows.net/hmlgreph/00212ti/01678/index.html>
4. Introdução ao middleware JDBC pela Dev Media. Disponível no endereço <https://www.devmedia.com.br/jdbc-tutorial/6638>. Acessado em 01/03/2023.
5. Introdução à Java Persistence API pela Dev Media. Disponível no endereço <https://www.devmedia.com.br/introducao-a-jpa-java-persistence-api/28173>. Acessado em 10/03/2023.
6. Introdução aos EJBs produzido pela Geek for Geeks. Disponível no endereço <https://www.geeksforgeeks.org/enterprise-java-beans-ejb/>. Acessado em 10/03/2023.
7. Bootstrap Docs. Disponível em <https://getbootstrap.com/docs/5.0/getting-started/introduction/>. Acessado em 10/03/2023.

Entrega da prática

Chegou a hora, gamer!

 Armazene o projeto em um repositório no GIT.

 Anexar a documentação do projeto (PDF) no GIT.

 Compartilhe o link do repositório do GIT com o seu tutor para correção da prática, por meio da **Sala de Aula Virtual**, na aba "**Trabalhos**" do respectivo nível de conhecimento.

 **Ei, verifique o prazo de entrega deste trabalho pois não aceitamos trabalhos fora do prazo!**